HiddenInk - Guide Technique (Edition Pro)

Document: Guide Technique
Version: 2.0 (Edition Pro)
Public: Developpeurs, mainteneurs, reviewers
Date: Avril 2026

Couverture

Ce guide documente l'architecture et les decisions techniques de HiddenInk:
- organisation des modules PNG/BMP/WAV;
- pipeline injection/extraction;
- points de build CMake/qmake;
- bonnes pratiques de maintenance et roadmap qualite.

Sommaire visuel

Couche	Role principal
Interfaces	CLI interactif/non-interactif + GUI Qt
Metier	Injection/extraction par format
Utilitaires	binaire, chiffrement XOR, histogramme, analyse
Build	CMake (CLI) + qmake (GUI)

Checklist mainteneur (avant merge)

- [ ] Build CMake propre (clean)
- [ ] Build qmake propre (Run qmake + build)
- [ ] Smoke tests PNG/BMP/WAV
- [ ] Verification analyse/histogramme
- [ ] Documentation utilisateur/installation/technique a jour

Encadre qualite

> Toute modification de chemins/sources GUI doit etre refletee dans `GUI/GUI.pro` puis validee par `Run qmake`.

1. Architecture générale

HiddenInk combine:
- une couche métier stéganographie (`stenographie/`)
- des interfaces CLI (`interfaces/cli/...`)
- une GUI Qt (`GUI/`)
- des utilitaires communs (`utils_bin`, chiffrement, histogramme, analyse)

Principes:
- découpage par format (PNG/BMP/WAV)
- logique de manipulation binaire centralisée autant que possible
- analyse séparée des opérations d'injection/extraction

2. Arborescence technique

```text
HiddenInk/
  CMakeLists.txt
  main.cpp
  interfaces/
    cli/interactif/
    cli/non_interactif/
  stenographie/
```

```
    PNG/
      png_hide_text.*
      png_hide_image.*
      png_extract.*
    BMP/
      bmp_convert.*
      bmp_Recuperation.*
    WAV/
      wav_hide.*
    utils/
      utils_bin.*
      encrypt/
        encrypt.*
      histogramme/
        histogram.*
      analysis/
        image_analysis.hpp
      external/
        stb_impl.cpp
  GUI/
    GUI.pro
    mainwindow.*
    histogramwidget.*
```

3. Pipeline de données

3.1 Injection (vue métier)

1. Lecture du média porteur
2. Lecture payload
3. Encodage binaire payload
4. Ajout signature/balises
5. Injection bit à bit (LSB / bit position configurée)
6. Sauvegarde sortie

3.2 Extraction

1. Lecture média stéganographié
2. Vérification signature
3. Recherche balises
4. Reconstruction flux binaire
5. Décodage payload (+ déchiffrement optionnel)
6. Détection type (magic bytes)
7. Sauvegarde sortie

4. Détails par format

4.1 PNG

- Cas supportés:
  - texte dans image
  - image dans image
- Paramètre critique: `bits/canal` (1-4 côté GUI)
- Compromis:
  - bits faibles = meilleure discrétion
  - bits élevés = meilleure capacité

4.2 BMP

- Injection/extraction de fichier dans bitmap
- Paramètre critique: `bitPos` (0-7)
- Support de clé hex optionnelle (XOR)

4.3 WAV

- Injection/extraction dans le chunk audio (`data`)
- Détection du chunk `data` via parsing RIFF
- Paramètre `bitPos` configurable
- Point d'attention:
  - inclure `<cstdint>` et utiliser `std::uint32_t` pour le parsing chunk size

5. Signatures, balises et utilitaires binaires

Fichier pivot: `stenographie/utils/utils_bin.*`

Fonctions clés:
- conversion texte/fichier -> binaire
- conversion binaire -> texte
- gestion signatures/balises
- vérification capacité du média

Convention logique:
- signature pour identifier payload HiddenInk
- balise d'ouverture / fermeture pour délimiter le contenu

6. Chiffrement (XOR)

Module: `stenographie/utils/encrypt/`

Capacités:
- génération de clé
- conversion hex <-> key
- XOR sur payload

Limite connue:
- XOR est une obfuscation simple, pas un chiffrement moderne robuste.

7. GUI Qt

7.1 Entrée principale

`GUI/mainwindow.cpp` construit les onglets:
- Injection
- Extraction
- Analyse
- Histogramme
- Outils
- Journal

7.2 Liens UI -> métier

Chaque bouton principal déclenche un slot `MainWindow::...`:
- `cacherTexte`, `cacherImage`, `cacherBMP`, `cacherWAV`
- `extraireTexte`, `extraireImage`, `extraireBMP`, `extraireWAV`
- analyse: `comparerImages`, `genererHistogramme`, `detecterStegano`, `analyserEntropie`, `detecterPayload`

La GUI agit comme orchestrateur:
- collecte paramètres;
- appelle API métier;
- journalise résultat;
- met à jour zone analyse/histogramme.

7.3 Object names importants

Les widgets sont retrouvés via `findChild` avec `objectName`.
Attention aux regressions si renommage UI:

- `editImageSourceTexte`, `editTexteFichier`, ...
- `histogramWidget`
- `analysisOutput`
- `logEdit`

8. Analyse et métriques

Module principal: `image_analysis.hpp` + logique GUI locale.

Fonctions mises en avant:
- MSE/PSNR
- histogramme (distribution)
- ratio LSB
- entropie Shannon
- payload detection (magic bytes)

9. Build et intégration

9.1 CMake (CLI)

Le `CMakeLists.txt` assemble l'exécutable `HiddenInk` avec les sources CLI/métier.

9.2 qmake (GUI)

`GUI/GUI.pro` inclut:
- sources GUI
- sources métiers externes via chemins relatifs

Points de vigilance:
- garder `SOURCES`/`HEADERS` synchronisés;
- relancer `qmake` après ajout de source;
- vérifier les chemins relatifs.

10. Qualité, test et validation

10.1 Stratégie minimale

- tests smoke manuels par format:
  - injecter -> extraire -> comparer
- validation visuelle + métrique (PSNR)
- validation payload (type détecté)

10.2 Cas limites

- capacité insuffisante
- mauvaise clé
- mauvais bit position
- fichiers corrompus / format invalide
- fichiers vides

10.3 Bonnes pratiques de maintenance

- privilégier des wrappers type-safe pour les conversions binaires;
- limiter la logique métier dans la GUI;
- centraliser les constantes de signatures/balises;
- documenter clairement les codes erreurs d'extraction.

11. Dette technique et améliorations proposées

Priorité haute:
- tests automatisés unitaires pour utilitaires binaires et parsing WAV;
- normalisation des codes de retour (enum/documentation centralisée);
- journalisation structurée (niveau, module, code).

Priorité moyenne:
- séparation plus stricte UI/service (MVC/MVVM léger);
- remplacement XOR par chiffrement moderne optionnel;
- export de rapport d'analyse (JSON/PDF).

Priorité basse:
- internationalisation UI (FR/EN);
- packaging cross-platform.

12. Procédure de release interne (suggestion)

1. Build clean CLI + GUI
2. Smoke tests injection/extraction PNG/BMP/WAV
3. Vérification analyse/histogramme
4. Génération docs PDF
5. Tag version + changelog

13. Références internes utiles

- `CMakeLists.txt`
- `GUI/GUI.pro`
- `GUI/mainwindow.cpp`
- `stenographie/WAV/wav_hide.cpp`
- `stenographie/utils/utils_bin.cpp`